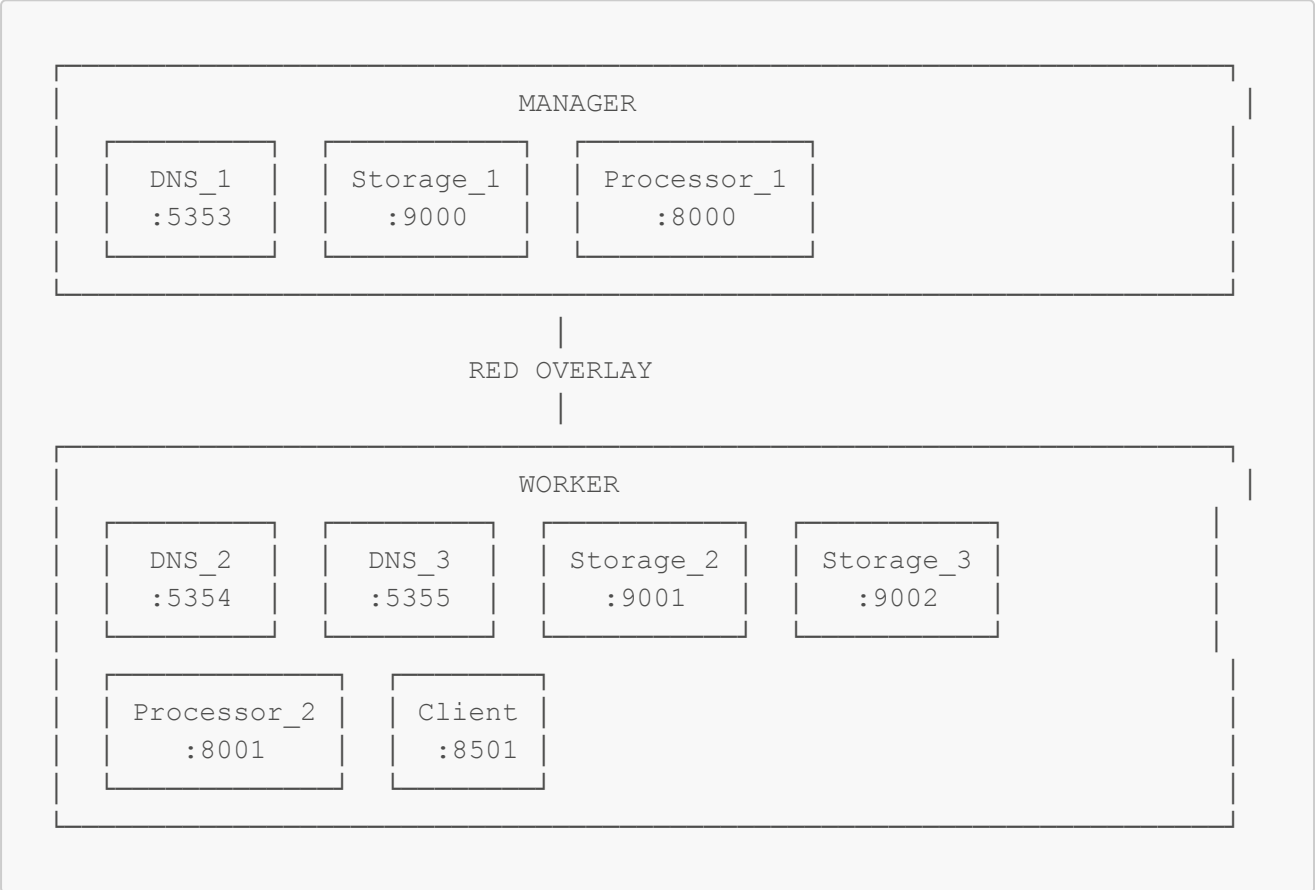
Despliegue Manual con Docker Run (Sin Docker Swarm Stack)

Este documento describe cómo desplegar la arquitectura completa usando `docker run` en lugar de Docker Swarm Stack. Útil para entender el sistema o para entornos donde no se puede usar Swarm.

Requisitos Previos

- **2 máquinas** en la misma red (Manager y Worker)
- Docker instalado en ambas máquinas
- Imágenes construidas en ambas máquinas

Arquitectura



PROF

Paso 1: Configuración Inicial

1.1 Obtener IPs de ambas máquinas

En el MANAGER:

```
hostname -I | awk '{print $1}'
# Ejemplo: 192.168.1.100
```

En el WORKER:

```
hostname -I | awk '{print $1}'
# Ejemplo: 192.168.1.101
```

> **Nota:** Guarda estas IPs, las usarás en los siguientes pasos.
>
> - `IP_MANAGER` = IP del Manager (ej: 192.168.1.100)
> - `IP_WORKER` = IP del Worker (ej: 192.168.1.101)

1.2 Crear directorios en AMBAS máquinas

```
# Crear directorios para volúmenes
sudo mkdir -p /srv/file-search/logs
sudo mkdir -p /srv/file-search/files

# Dar permisos
sudo chmod -R 777 /srv/file-search

# Copiar archivos de ejemplo (solo si tienes archivos para indexar)
# cp -r /ruta/a/tus/archivos/* /srv/file-search/files/
```

Paso 2: Inicializar Docker Swarm (Solo para la red overlay)

Aunque no usaremos el stack, necesitamos Swarm para crear una red overlay entre las máquinas.

2.1 En el MANAGER - Inicializar Swarm

PROF

```
docker swarm init --advertise-addr <IP_MANAGER>

# Ejemplo:
# docker swarm init --advertise-addr 192.168.1.100
```

Guarda el comando que aparece, será algo como:

```
docker swarm join --token SWMTKN-1-xxxxx <IP_MANAGER>:2377
```

2.2 En el WORKER - Unirse al Swarm

```
docker swarm join --token SWMTKN-1-xxxxx <IP_MANAGER>:2377
```

2.3 En el MANAGER - Crear la red overlay

```
docker network create \
  --driver overlay \
  --attachable \
  file_search_net
```

> **Importante:** La opción `--attachable` permite que contenedores iniciados con `docker run` se conecten a la red overlay.

Paso 3: Construir las Imágenes

3.1 En el MANAGER - Construir todas las imágenes

```
cd /ruta/al/proyecto/file-search

# DNS Service
docker build -t file-search-dns:latest -f app/dns_service/Dockerfile .

# Storage
docker build -t file-search-storage:latest -f Dockerfile.storage .

# Processor
docker build -t file-search-processor:latest -f Dockerfile.processor .

# Client
docker build -t file-search-client:latest -f Dockerfile.client .
```

PROF

3.2 Transferir imágenes al WORKER

En el MANAGER:

```
# Exportar todas las imágenes
docker save \
  file-search-dns:latest \
  file-search-storage:latest \
  file-search-processor:latest \
  file-search-client:latest \
  -o /tmp/file-search-images.tar

# Copiar al worker
scp /tmp/file-search-images.tar usuario@<IP_WORKER>:/tmp/
```

En el WORKER:

```
# Cargar las imágenes
docker load -i /tmp/file-search-images.tar
```

Paso 4: Iniciar Contenedores en el MANAGER

4.1 DNS_1 (Puerto 5353)

```
docker run -d \
  --name dns_1 \
  --hostname dns_1 \
  --network file_search_net \
  --network-alias dns \
  -p 5353:5353 \
  -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs \
  -e LOG_LEVEL=INFO \
  -e DNS_SERVER_ID=dns_1 \
  -e DNS_PORT=5353 \
  -e DNS_ALIAS=dns \
  -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 \
  -e DISCOVERY_INTERVAL=15 \
  -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest
```

4.2 Storage_1 (Puerto 9000)

PROF

```
docker run -d \
  --name storage_1 \
  --hostname storage_1 \
  --network file_search_net \
  --network-alias storage \
  -p 9000:8000 \
  -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_1 \
  -e STORAGE_PORT=8000 \
  -e SERVER_ID=storage_1 \
  -e SERVER_PORT=8000 \
  -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files \
  -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db \
```

```
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e HEARTBEAT_INTERVAL=5 \
  -e SYNC_INTERVAL=10 \
  file-search-storage:latest
```

4.3 Processor_1 (Puerto 8000)

```
docker run -d \
  --name processor_1 \
  --hostname processor_1 \
  --network file_search_net \
  --network-alias processor \
  -p 8000:8000 \
  -v /srv/file-search/logs:/app/logs \
  -e PROCESSOR_ID=processor_1 \
  -e PROCESSOR_PORT=8000 \
  -e
STORAGE_NODES=http://storage_1:8000,http://storage_2:8000,http://storage
_3:8000 \
  -e STORAGE_URL=http://storage_1:8000 \
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e STORAGE_TIMEOUT=10 \
  -e STORAGE_MAX_RETRIES=3 \
  -e STORAGE_RETRY_DELAY=0.5 \
  -e LOG_DIR=/app/logs \
  file-search-processor:latest
```

Paso 5: Iniciar Contenedores en el WORKER

PROF

5.1 DNS_2 (Puerto 5354)

```
docker run -d \
  --name dns_2 \
  --hostname dns_2 \
  --network file_search_net \
  --network-alias dns \
  -p 5354:5353 \
  -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs \
  -e LOG_LEVEL=INFO \
  -e DNS_SERVER_ID=dns_2 \
  -e DNS_PORT=5353 \
  -e DNS_ALIAS=dns \
  -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 \
```

```
  -e DISCOVERY_INTERVAL=15 \
  -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest
```

5.2 DNS_3 (Puerto 5355)

```
docker run -d \
  --name dns_3 \
  --hostname dns_3 \
  --network file_search_net \
  --network-alias dns \
  -p 5355:5353 \
  -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs \
  -e LOG_LEVEL=INFO \
  -e DNS_SERVER_ID=dns_3 \
  -e DNS_PORT=5353 \
  -e DNS_ALIAS=dns \
  -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 \
  -e DISCOVERY_INTERVAL=15 \
  -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest
```

5.3 Storage_2 (Puerto 9001)

```
docker run -d \
  --name storage_2 \
  --hostname storage_2 \
  --network file_search_net \
  --network-alias storage \
  -p 9001:8000 \
  -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_2 \
  -e STORAGE_PORT=8000 \
  -e SERVER_ID=storage_2 \
  -e SERVER_PORT=8000 \
  -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files \
  -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db \
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e HEARTBEAT_INTERVAL=5 \
```

PROF

```
  -e SYNC_INTERVAL=10 \
  file-search-storage:latest
```

5.4 Storage_3 (Puerto 9002)

```
docker run -d \
  --name storage_3 \
  --hostname storage_3 \
  --network file_search_net \
  --network-alias storage \
  -p 9002:8000 \
  -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_3 \
  -e STORAGE_PORT=8000 \
  -e SERVER_ID=storage_3 \
  -e SERVER_PORT=8000 \
  -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files \
  -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db \
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e HEARTBEAT_INTERVAL=5 \
  -e SYNC_INTERVAL=10 \
  file-search-storage:latest
```

5.5 Processor_2 (Puerto 8001)

```
docker run -d \
  --name processor_2 \
  --hostname processor_2 \
  --network file_search_net \
  --network-alias processor \
  -p 8001:8000 \
  -v /srv/file-search/logs:/app/logs \
  -e PROCESSOR_ID=processor_2 \
  -e PROCESSOR_PORT=8000 \
  -e
STORAGE_NODES=http://storage_1:8000,http://storage_2:8000,http://storage
_3:8000 \
  -e STORAGE_URL=http://storage_1:8000 \
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e STORAGE_TIMEOUT=10 \
  -e STORAGE_MAX_RETRIES=3 \
  -e STORAGE_RETRY_DELAY=0.5 \
```

PROF

```
  -e LOG_DIR=/app/logs \
  file-search-processor:latest
```

5.6 Client (Puerto 8501)

```
docker run -d \
  --name client \
  --hostname client \
  --network file_search_net \
  -p 8501:8501 \
  -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 \
  -e TARGET_SERVICE_NAME=processor \
  -e TARGET_SERVICE_PORT=8000 \
  -e API_BASE_URL=http://processor_1:8000 \
  -e BROWSER_API_URL=http://<IP_MANAGER>:8000 \
  -e BROWSER_API_URL_WORKER=http://<IP_WORKER>:8001 \
  -e MAX_RETRIES=3 \
  -e RETRY_DELAY=0.5 \
  file-search-client:latest
```

> **Nota:** Reemplaza `<IP_MANAGER>` y `<IP_WORKER>` con las IPs reales.
>
> - `BROWSER_API_URL` se usa para descargas cuando el Manager está disponible
> - `BROWSER_API_URL_WORKER` es el fallback cuando hay partición de red y solo el Worker está disponible

---

Paso 6: Verificación

6.1 En el MANAGER - Verificar contenedores locales

PROF

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Deberías ver:

```
NAMES         STATUS          PORTS
dns_1         Up X minutes    0.0.0.0:5353->5353/tcp
storage_1     Up X minutes    0.0.0.0:9000->8000/tcp
processor_1   Up X minutes    0.0.0.0:8000->8000/tcp
```

6.2 En el WORKER - Verificar contenedores locales

```
```

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Deberías ver:

```
NAMES         STATUS          PORTS
dns_2         Up X minutes    0.0.0.0:5354->5353/tcp
dns_3         Up X minutes    0.0.0.0:5355->5353/tcp
storage_2     Up X minutes    0.0.0.0:9001->8000/tcp
storage_3     Up X minutes    0.0.0.0:9002->8000/tcp
processor_2   Up X minutes    0.0.0.0:8001->8000/tcp
client        Up X minutes    0.0.0.0:8501->8501/tcp
```

6.3 Probar conectividad entre contenedores

Desde el MANAGER:

```
# Probar que dns_1 puede ver a dns_2
docker exec dns_1 ping -c 2 dns_2

# Probar que processor_1 puede ver a storage_2
docker exec processor_1 ping -c 2 storage_2
```

6.4 Verificar servicios DNS

```
# Desde cualquier máquina
curl http://<IP_MANAGER>:5353/health
curl http://<IP_WORKER>:5354/health

# Ver processors registrados
curl http://<IP_MANAGER>:5353/processor/list

# Ver storages registrados
curl http://<IP_MANAGER>:5353/server/list
```

PROF

6.5 Probar la aplicación

Abre en tu navegador:

```
http://<IP_WORKER>:8501
```

O desde el Manager:

```
http://<IP_MANAGER>:8501  # Si el cliente está ahí
```

Paso 7: Comandos Útiles

Ver logs de un contenedor

```
docker logs -f <nombre_contenedor>

# Ejemplos:
docker logs -f dns_1
docker logs -f processor_1 --tail 50
```

Reiniciar un contenedor

```
docker restart <nombre_contenedor>
```

Detener todos los contenedores

En el MANAGER:

```
docker stop dns_1 storage_1 processor_1
docker rm dns_1 storage_1 processor_1
```

En el WORKER:

PROF

```
docker stop dns_2 dns_3 storage_2 storage_3 processor_2 client
docker rm dns_2 dns_3 storage_2 storage_3 processor_2 client
```

Eliminar la red (después de detener todos los contenedores)

En el MANAGER:

```
docker network rm file_search_net
```

Salir del Swarm

En el WORKER:

```
docker swarm leave
```

En el MANAGER:

```
docker swarm leave --force
```

Troubleshooting

Error: "network file_search_net not found"

La red overlay solo se propaga cuando hay contenedores usándola. Asegúrate de:

1. Crear la red en el Manager primero
2. Iniciar al menos un contenedor en el Manager antes de iniciar contenedores en el Worker

Error: "Cannot connect to container on other node"

Verifica que:

1. Ambos nodos están en el mismo Swarm (`docker node ls` en el Manager)
2. Los puertos del Swarm están abiertos (2377, 7946, 4789)
3. La red es overlay y attachable

```
# Verificar la red
docker network inspect file_search_net
```

Contenedor no puede resolver nombres DNS internos

PROF

Los contenedores deben estar en la misma red overlay. Verifica:

```
docker inspect <contenedor> | grep -A 20 "Networks"
```

Ver qué contenedores están en la red

```
docker network inspect file_search_net --format '{{range .Containers}}
{{.Name}} {{end}}'
```

Script de Inicio Rápido

Para el MANAGER (guardar como `start-manager.sh`):

```
#!/bin/bash
set -e

echo "=== Iniciando servicios en MANAGER ==="

# DNS_1
docker run -d --name dns_1 --hostname dns_1 \
  --network file_search_net --network-alias dns \
  -p 5353:5353 -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs -e LOG_LEVEL=INFO -e DNS_SERVER_ID=dns_1 \
  -e DNS_PORT=5353 -e DNS_ALIAS=dns -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 -e DISCOVERY_INTERVAL=15 -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest

echo "DNS_1 iniciado"
sleep 2

# Storage_1
docker run -d --name storage_1 --hostname storage_1 \
  --network file_search_net --network-alias storage \
  -p 9000:8000 -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_1 -e STORAGE_PORT=8000 -e SERVER_ID=storage_1 \
  -e SERVER_PORT=8000 -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 -e HEARTBEAT_INTERVAL=5 -e SYNC_INTERVAL=10 \
  file-search-storage:latest

echo "Storage_1 iniciado"
sleep 2

# Processor_1
docker run -d --name processor_1 --hostname processor_1 \
  --network file_search_net --network-alias processor \
  -p 8000:8000 -v /srv/file-search/logs:/app/logs \
  -e PROCESSOR_ID=processor_1 -e PROCESSOR_PORT=8000 \
  -e
STORAGE_NODES=http://storage_1:8000,http://storage_2:8000,http://storage
_3:8000 \
  -e STORAGE_URL=http://storage_1:8000 -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 -e STORAGE_TIMEOUT=10 \
  -e STORAGE_MAX_RETRIES=3 -e STORAGE_RETRY_DELAY=0.5 -e
LOG_DIR=/app/logs \
  file-search-processor:latest

echo "Processor_1 iniciado"
```

```
echo "=== MANAGER listo ==="
docker ps
```

Para el WORKER (guardar como `start-worker.sh`):

```
#!/bin/bash
set -e

MANAGER_IP="${1:-192.168.1.100}"  # Pasar IP del manager como argumento

echo "=== Iniciando servicios en WORKER ==="
echo "Manager IP: $MANAGER_IP"

# DNS_2
docker run -d --name dns_2 --hostname dns_2 \
  --network file_search_net --network-alias dns \
  -p 5354:5353 -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs -e LOG_LEVEL=INFO -e DNS_SERVER_ID=dns_2 \
  -e DNS_PORT=5353 -e DNS_ALIAS=dns -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 -e DISCOVERY_INTERVAL=15 -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest

echo "DNS_2 iniciado"

# DNS_3
docker run -d --name dns_3 --hostname dns_3 \
  --network file_search_net --network-alias dns \
  -p 5355:5353 -v /srv/file-search/logs:/app/logs \
  -e LOG_DIR=/app/logs -e LOG_LEVEL=INFO -e DNS_SERVER_ID=dns_3 \
  -e DNS_PORT=5353 -e DNS_ALIAS=dns -e HEALTH_CHECK_INTERVAL=5 \
  -e SYNC_INTERVAL=5 -e DISCOVERY_INTERVAL=15 -e API_SERVER_TIMEOUT=30 \
  -e PROCESSOR_TIMEOUT=30 \
  file-search-dns:latest

echo "DNS_3 iniciado"
sleep 2

# Storage_2
docker run -d --name storage_2 --hostname storage_2 \
  --network file_search_net --network-alias storage \
  -p 9001:8000 -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_2 -e STORAGE_PORT=8000 -e SERVER_ID=storage_2 \
  -e SERVER_PORT=8000 -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 -e HEARTBEAT_INTERVAL=5 -e SYNC_INTERVAL=10 \
  file-search-storage:latest
```

PROF

```
echo "Storage_2 iniciado"

# Storage_3
docker run -d --name storage_3 --hostname storage_3 \
  --network file_search_net --network-alias storage \
  -p 9002:8000 -v /srv/file-search/files:/tmp/source_files:ro \
  -v /srv/file-search/logs:/app/logs \
  -e STORAGE_ID=storage_3 -e STORAGE_PORT=8000 -e SERVER_ID=storage_3 \
  -e SERVER_PORT=8000 -e FILES_ROOT=/app/files \
  -e FILES_SOURCE_PATH=/tmp/source_files -e LOG_DIR=/app/logs \
  -e DB_PATH=/app/data/storage.db -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 -e HEARTBEAT_INTERVAL=5 -e SYNC_INTERVAL=10 \
  file-search-storage:latest

echo "Storage_3 iniciado"
sleep 2

# Processor_2
docker run -d --name processor_2 --hostname processor_2 \
  --network file_search_net --network-alias processor \
  -p 8001:8000 -v /srv/file-search/logs:/app/logs \
  -e PROCESSOR_ID=processor_2 -e PROCESSOR_PORT=8000 \
  -e
STORAGE_NODES=http://storage_1:8000,http://storage_2:8000,http://storage
_3:8000 \
  -e STORAGE_URL=http://storage_1:8000 -e DNS_ALIAS=dns \
  -e DNS_SERVICE_PORT=5353 -e STORAGE_TIMEOUT=10 \
  -e STORAGE_MAX_RETRIES=3 -e STORAGE_RETRY_DELAY=0.5 -e
LOG_DIR=/app/logs \
  file-search-processor:latest

echo "Processor_2 iniciado"

# Client
docker run -d --name client --hostname client \
  --network file_search_net \
  -p 8501:8501 \
  -e DNS_ALIAS=dns -e DNS_SERVICE_PORT=5353 \
  -e TARGET_SERVICE_NAME=processor -e TARGET_SERVICE_PORT=8000 \
  -e API_BASE_URL=http://processor_1:8000 \
  -e BROWSER_API_URL=http://${MANAGER_IP}:8000 \
  -e MAX_RETRIES=3 -e RETRY_DELAY=0.5 \
  file-search-client:latest

echo "Client iniciado"
echo "=== WORKER listo ==="
docker ps
```

PROF

Uso:

```
# En el Manager
chmod +x start-manager.sh
./start-manager.sh

# En el Worker
chmod +x start-worker.sh
./start-worker.sh 192.168.1.100  # IP del Manager
```

Resumen de Puertos

Servicio	Máquina	Puerto Host	Puerto Contenedor
DNS_1	Manager	5353	5353
DNS_2	Worker	5354	5353
DNS_3	Worker	5355	5353
Storage_1	Manager	9000	8000
Storage_2	Worker	9001	8000
Storage_3	Worker	9002	8000
Processor_1	Manager	8000	8000
Processor_2	Worker	8001	8000
Client	Worker	8501	8501

Acceso a la Aplicación

- **Cliente Web**: `http://<IP_WORKER>:8501`
- **API (Processor 1)**: `http://<IP_MANAGER>:8000`
- **API (Processor 2)**: `http://<IP_WORKER>:8001`

PROF